2019112602 에너지신소재공학과 양정모

Q1 김풍 작가의 '조립식 요리'를 'OOP의 장점'으로 해석해봅시다.

```java
public class ChiliOil {
    public String flavor = "매운맛";
    public int spiciness = 10; // 맵기 정도

    public void sizzle() {
        System.out.println("(고추기름 향 보글보글)");
    }
}
```

```java
class Jjamppong {
    String name = "매운 짬뽕";
    ChiliOil oil; // 고추기름 부품을 담을 공간

    void cook(ChiliOil oil) {
        this.oil = oil; // 전달받은 고추기름 조립
        System.out.print("[" + name + "] 제작 중... ");
        oil.sizzle();
    }
}
```

```java
class MapoTofu {
    String name = "매운 마파두부";
    ChiliOil oil;

    void cook(ChiliOil oil) {
        this.oil = oil; // 동일한 고추기름 부품 재사용
        System.out.print("[" + name + "] 제작 중... ");
        oil.sizzle();
    }
}
```

```java
public class KimPoong {
    public static void main(String[] args) {
        // 1. 일단 고추기름 설계도로 실제 '고추기름 객체'를 하나 딱 하나 만들기(인스턴스화)
        ChiliOil mySecretOil = new ChiliOil();

        // 2. 요리 시작
        Jjamppong jp = new Jjamppong();
        MapoTofu mt = new MapoTofu();

        System.out.println("김풍 작가의 '15분 요리' 시작합니다!\n");

        // 3. 하나의 고추기름 객체를 여기저기 '조립'해서 사용함(재사용성)
        jp.cook(mySecretOil);
        mt.cook(mySecretOil);

        System.out.println("\n고추기름 하나로 모든 요리가 완성되었습니다!");
    }
}
```

```java
public class KimPoong {
        public static void main(String[] args) {
                //1. 일단 고추기름 설계도로 실제 '고추기름 객체'를 하나 딱 하나 만들기(인스턴스화)
                ChiliOil mySecretOil = new ChiliOil();

                //2. 요리 시작
                Jjamppong jp = new Jjamppong();
                MapoTofu mt = new MapoTofu();

                System.out.println("김풍 작가의 '15 분 요리' 시작합니다!\n");

                //3. 하나의 고추기름 객체를 여기저기 '조립'해서 사용함(재사용성)
                jp.cook(mySecretOil);
                mt.cook(mySecretOil);

                System.out.println("\n 고추기름 하나로 모든 요리가 완성되었습니다!");
        }
}


//인스턴스화가 참 중요하다~
//getter 와 setter : 캡슐화를 통해서 보안을 용이하게 하고 중요한 정보를 숨긴다
//실습 2 번에서 캡슐화가 게터 세터를 설명한다
//this 문법은 이해 못했을 거라고 하면서 설명
//
public class ChiliOil {
        public String flavor = "매운맛";
        public int spiciness = 10; //맵기 정도

        public void sizzle() {
                System.out.println("(고추기름 향 보글보글)");
```

```java
        }
}


public class Jjamppong {
        String name = "매운 짬뽕";
        ChiliOil oil; //고추기름 부품을 담을 공간

        void cook(ChiliOil oil) {
                this.oil = oil; //전달받은 고추기름 조립
                System.out.print("[" + name + "] 제작 중 ... ");
                oil.sizzle();
        }
}


public class MapoTofu {
        String name = "매운 마파두부";
        ChiliOil oil;

        void cook(ChiliOil oil) {
                this.oil = oil;//동일한 고추기름 부품 사용
                System.out.print("[" + name + "] 제작 중 ...");
                oil.sizzle();
        }
}
```

Q2 학점을 조작하려는 해커로부터 private와 setter로 값을 보호해 봅시다.

```java
public class Student {
    // 1. 필드는 외부에서 직접 수정 불가능하도록 private으로 선언
    private String name;
    private int studentId;
    private double gpa; // 학점 (0.0 ~ 4.5)

    // 2. 생성자
    public Student(String name, int studentId)
    {
        this.name = name;
        this.studentId = studentId;
    }

    // 3. getter: 외부에서 안전하게 읽기만 가능
    public String getName() { return name; }
    public int getStudentId() { return studentId; }
    public double getGpa() { return gpa; }

    // 4. setter: 데이터 검증 로직 포함
    public void setGpa(double gpa) {
        // 0.0 미만이거나 4.5를 초과하는 학점은 저장을 거부함
        if (gpa < 0.0 || gpa > 4.5)
        {
            System.err.println("잘못된 학점 입력입니다. (입력값: " + gpa + ")");
            return; // 메소드 종료 (값 저장 안 됨)
        }
        this.gpa = gpa; // 유효한 경우에만 필드에 저장
    }
}
```

```java
public class AttackSystem {
    public static void main(String[] args) {
        Student s1 = new Student("임다영", 12485007);

        // 공격 1: 외부에서 직접 수정 시도
        //s1.gpa = 4.5;

        // 공격 2: 잘못된 데이터 입력 시도
        s1.setGpa(9.9);
    }
}
```

```java
public class Student {
        //1. 필드는 외부에서 직접 수정이 불가능하도록 private 으로 선언
        private String name;
        private int studentID;
        public double gpa; //학점 (0.0~4.5)

        //2. 생성자
        public Student(String name, int studentID)
        {
                this.name = name;
                this.studentID = studentID;
        }

        //3. getter : 외부에서 안전하게 읽기만 가능
        public String getName() { return name; }
        public int getStudentID() { return studentID; }
        public double getGpa() { return gpa; }

        //4. setter : 데이터 검증 로직 포함
        public void setGpa(double gpa) {
                //0.0 미만익나 4.5 초과하는 학점은 저장을 거부함
                if (gpa < 0.0 || gpa > 4.5)
                {
                        System.err.println("잘못된 학점 입력입니다. (입력값: " + gpa +
")");
                        return; //메소드 종료 (값 저장 안됨)
                }
                this.gpa = gpa; //유효한 경우에만 필드에 저장
        }
}


public class AttackSystem {
        public static void main(String[] args) {
                Student s1 = new Student("임다영", 12485007);

                //공격 1:외부에서 직접 수정 시도
                s1.gpa = 4.5;
```

```
                //공격 2 : 잘못된 데이터 입력 시도
                //s1.setGpa(9.9);
        }
}
```

```
Problems  @ Javadoc  Declaration  Console ×  Eclipse IDE for Java Developers 2026-03 Release
<terminated> AttackSystem [Java Application] C:₩Users₩master₩.p2₩pool₩plugins₩org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20
잘못된 학점 입력입니다. (입력값: 9.9)
```

Q3 연등제 준비를 위해 연등을 인스턴스화 하여 학교에 설치해 봅시다.

```java
public class LotusLantern {
    public String color;  // 연등 색상
    public String shape;  // 연등 모양
    public String wish;   // 적힌 소원

    // 기본 색상과 모양만 정하는 생성자
    public LotusLantern(String color, String shape) {
        // this()는 반드시 첫 줄에 와야 함
        this(color, shape, "연애하게 해주세요");
    }

    // 모든 정보를 지정하는 생성자 (실제 초기화 담당)
    public LotusLantern(String color, String shape, String wish) {
        // this.필드명 = 매개변수명 으로 구분
        this.color = color;
        this.shape = shape;
        this.wish = wish;
    }

    public void lightUp() {
        System.out.printf("[%s %s 연등]이 빛납니다 반짝반짝~ 소원: \"%s\" %n", color, shape, wish);
    }
}
```

```java
public class DonggukFestival {
    public static void main(String[] args) {
        System.out.println("동국대학교 연등제 시즌 입니다.\n");

        // 1. 인자가 2개인 생성자 호출 (내부적으로 this()를 통해 3개짜리 호출)
        LotusLantern lantern1 = new LotusLantern("빨간색", "연꽃 모양");

        // 2. 인자가 3개인 생성자 호출 (직접 소원 기입)
        LotusLantern lantern2 = new LotusLantern("분홍색", "팔각 모양", "자바 중간고사 100점");

        // 결과 확인
        lantern1.lightUp();
        lantern2.lightUp();
    }
}
```

```java
public class LotusLantern {
        public String color; //연등 색상
        public String shape; //연등 모양
        public String wish; //적힌 소원

        //기본 색상과 모양만 정하는 생성기
```

```java
        public LotusLantern(String color, String shape) {
                //this()는 반드시 첫 줄에 와야 함
                this(color, shape, "연애하게 해주세요");
        }

        //모든 정보를 저장하는 생성자 (실제 초기화 담당)
        public LotusLantern(String color, String shape, String wish) {
                //this.필드명 = 매개변수명 으로 구분
                this.color = color;
                this.shape = shape;
                this.wish = wish;
        }

        public void lightUp() {
                System.out.printf("[%s %s 연등]이 빛납니다 반짝반짝~ 소원: \"%s\" %n",
color, shape, wish);
        }
}


public class DonggukFestival {
        public static void main(String[] args) {
                System.out.println("동국대학교 연등제 시즌입니다. \n");

                //1. 인자가 2개인 생성자 흐름 (내부적으로 this()를 통해 3개짜리 호출)
                LotusLantern lantern1 = new LotusLantern("빨간색", "연꽃 모임");

                //2. 인자가 3개인 생성자 호출 (직접 소원 기입)
                LotusLantern lantern2 = new LotusLantern("분홍색", "팔각 모양", "자바
중간고사 100점");

                //결과 확인
                lantern1.lightUp();
                lantern2.lightUp();
        }
}
```

```
Problems  @ Javadoc  Declaration  Console ×  Eclipse IDE for Java Develop
<terminated> DonggukFestival [Java Application] C:₩Users₩master₩.p2₩pool₩plugins₩org.ec
동국대학교 연등제 시즌입니다.

[빨간색 연꽃 모임 연등]이 빛납니다 반짝반짝~ 소원: "연애하게 해주세요"
[분홍색 팔각 모양 연등]이 빛납니다 반짝반짝~ 소원: "자바 중간고사 100점"
```